

Security Assessment Report

Contract: `contracts/TroyToken.sol`

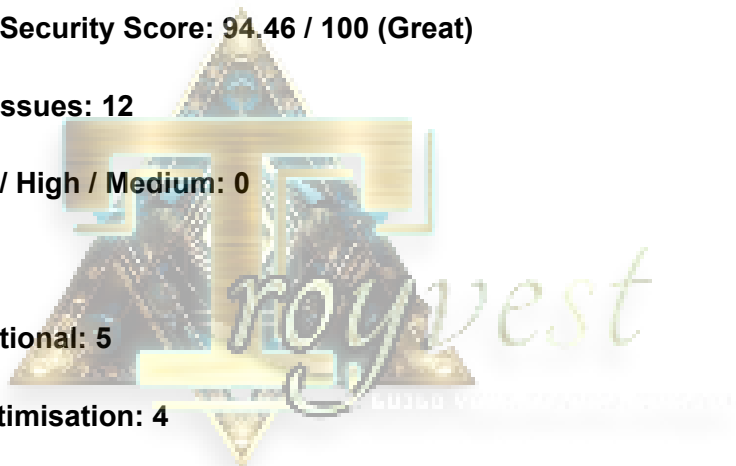
Network Target: BSC-compatible(Solidity / EVM)

Scan Tool: SolidityScan (static analysis)

Scan Date: 18Nov 2025

1. Executive Summary

The `TroyToken.sol` smart contract was analysed using automated static analysis to identify security, reliability, and gas-efficiency issues.

- **Overall Security Score: 94.46 / 100 (Great)**
 - **Total Issues: 12**
 - **Critical / High / Medium: 0**
 - **Low: 3**
 - **Informational: 5**
 - **Gas Optimisation: 4**
- 
- A large, semi-transparent watermark of the TroyVest logo is centered over the list. The logo features a stylized 'T' and 'V' in a blue and gold color scheme, with the word 'troyvest' in a gold script font below it. The background of the logo is a dark, textured blue.

No vulnerabilities were found that pose a direct risk to user funds or system integrity. All identified issues are minor and can be addressed without major architectural changes.

The contract is **considered suitable for deployment**, assuming recommended low-severity and best-practice fixes are implemented.

2. Vulnerability Classification & Severity

Issues are grouped by their potential impact:

- **Critical:** May lead to direct loss or permanent lock of funds, or total contract compromise.

- **High:** Can severely affect contract logic, access control, or economic behaviour.
- **Medium:** Affects correctness or business rules but typically requires specific conditions.
- **Low:** Minor deviations from best practices; limited impact, but worth fixing.
- **Informational:** Style, documentation, or maintainability concerns.
- **Gas:** Inefficiencies that increase gas usage but do not affect safety.

No Critical, High, or Medium issues were reported in this assessment.

3. Findings Summary

Severity	Count	Notes
Critical	0	None
High	0	None
Medium	0	None
Low	3	Minor best-practice/security hygiene issues
Informational	5	NatSpec / style / documentation
Gas	4	Micro gas optimisations

4. Detailed Findings

Below are the individual issues as grouped by severity. IDs and wording are aligned with the SolidityScan output and paraphrased for clarity.

4.1 Low-Severity Issues

L001 – Use of FloatingPragma

- **Severity:** Low

- **Description:**
The contract uses a version range (e.g. ^0.8.x) instead of a fixed compiler version.
This can cause slightly different behaviour if compiled with future compiler releases.
- **Risk:**
Low. Does not create an immediate exploit, but can introduce inconsistency between builds.

Recommendation:

Use a fixed, audited compiler version, for example:

```
pragma solidity 0.8.26;
```

-

L002 – Outdated Compiler Version

- **Severity:** Low
- **Description:**
The compiler version specified is older than the latest stable in the 0.8.x line. Newer versions often include minor security and correctness improvements.
- **Risk:**
Low. The contract remains functional but may miss recent safety checks and optimisations.
- **Recommendation:**
Upgrade to a recent stable version (e.g. 0.8.26) and re-run the tests and audit.

L003 – Use Ownable2Step for Ownership Transfers

- **Severity:** Low
- **Description:**
The contract uses single-step ownership transfer. If the new owner address is mis-typed, ownership can be irreversibly lost.

- **Risk:**
Organisational / operational risk rather than a user-funds risk.
 - **Recommendation:**
Use OpenZeppelin's [Ownable2Step](#) pattern to require the new owner to explicitly accept ownership, reducing the risk of error.
-

4.2 Informational Issues

These do not affect security or runtime behaviour, but improve clarity and auditability.

I001 – Missing [@author](#) in NatSpec Comments

- **Severity:** Informational
 - **Description:**
Contract header comments do not identify the author/owner of the code.
 - **Recommendation:**
Add a NatSpec [@author](#) tag in the main contract documentation.
-

I002 – Missing NatSpec Summary/Notice for Contract Declaration

- **Severity:** Informational
 - **Description:**
The main contract declaration lacks a clear [@title](#) / [@notice](#) summarising its purpose.
 - **Recommendation:**
Add high-level NatSpec comments to clearly define the contract's role (e.g. fixed-supply BEP-20 token for the Troy ecosystem).
-

I003 – Missing [@inheritdoc](#) on Overridden Functions

- **Severity:** Informational
 - **Description:**
Some functions that override parent contract methods (e.g. `ERC20` / `Ownable`) do not use `@inheritdoc`.
 - **Recommendation:**
Use `@inheritdoc` for overridden functions to inherit base documentation and keep the interface clear.
-

I004 – Missing NatSpec Comments for Constructors

- **Severity:** Informational
 - **Description:**
The constructor lacks detailed NatSpec explaining parameters and allocation logic.
 - **Recommendation:**
Document constructor parameters, distribution rules, and any assumptions (e.g. sum of allocations equals total supply).
-

I005 – Use Scientific Notation for Large Constants

- **Severity:** Informational
- **Description:**
Large numeric constants, such as total supply, are not expressed in a standardised scientific or underscore-separated format.

Recommendation:

Use either underscores or scientific-style notation for clarity, e.g.:

```
uint256 public constant MAX_SUPPLY = 888_888_888 * 1e18;
```

-
-

4.3 Gas Optimisation Issues

These issues do **not** represent vulnerabilities; they simply suggest ways to reduce gas consumption.

G001 – Constructor Not Marked Payable

- **Severity:** Gas
 - **Description:**
The constructor could be declared **payable** to slightly reduce deployment cost when no Ether is transferred.
 - **Impact:**
Minor gas saving at deployment time.
 - **Recommendation:**
Mark the constructor as **payable** and, if applicable, forward any attached native currency to a treasury address.
-

G002 – Long **require** / **revert** Strings (3 Instances)

- **Severity:** Gas
- **Description:**
Error messages used in **require**/ **revert** are longer than necessary. Long strings increase bytecode size and gas usage.
- **Impact:**
Slightly higher deployment and execution gas costs.

Recommendation:

Replace verbose strings with short codes or abbreviations, and document them in external docs.

```
require(amount > 0, "ERR_AMT");
```

-

(Other gas notices in the report fall under the same category of message length and style.)

5. Scan History

#	Date	Security Score	Critical	High	Medium	Low	Info	Gas
1	2025-11-18	94.46	0	0	0	3	5	4

Only a single scan run is recorded; no regressions or worsening of the risk profile have been observed.

6. Overall Conclusion

- The **TroyToken.sol** contract demonstrates **strong security hygiene** with **no Critical, High, or Medium-severity** findings.
- All identified issues are **minor** and relate to version pinning, documentation, and gas optimisations.
- With the recommended fixes applied, the contract will present an even stronger posture for investors and community review.

Final Judgement:

The contract is **fit for deployment**, provided low-severity and best-practice recommendations are implemented and a final re-scan / manual review confirms there are no regressions.

7. General Disclaimer

- This audit is based on the specific version of **TroyToken.sol** that was scanned on **18 Nov 2025**. Any subsequent code changes are outside the scope of this report.
- Static analysis tools can only detect patterns they are designed to recognise. Hidden logic risks, economic design flaws, or governance misuse may still exist.

- This report is **not financial advice** and does not guarantee the future performance, price, or legal compliance of the token or associated project.
- A defence-in-depth approach is strongly recommended: internal reviews, bug bounty programmes, on-chain monitoring, and periodic re-audits, especially after upgrades.

If you'd like, I can now:

- Turn this into a **formatted PDF-style text** (headings + logo placeholders)
- Or write a **1-page “Security Highlights for Investors”** version for your website / whitepaper.

